

Meeting Assistant Roadmap & Future Work

Document 3 of 3 · Product roadmap, advanced capabilities, and research directions · [Overview](#) · [Implementation](#) · [Changelog](#)

This document consolidates everything that comes *after* the MVP: the sprint-by-sprint product roadmap, the three advanced differentiating capabilities (facilitator guide, document alignment, sentiment), the operational quality loop (MLOps), and a set of longer-horizon research ideas. It is the "future ideas" companion to [Document 1](#) (vision) and [Document 2](#) (the system as built today).

Status legend. Sprint 1 (the MVP) is complete. Within the advanced capabilities, the **facilitator guide** is already implemented (`GET /api/meetings/{id}/facilitation`); document alignment and sentiment analysis are designed but not yet built.

Permanently out of scope: live streaming speech-to-text during a meeting and professional speaker diarization. Single-shot Gemini transcription of uploaded/recorded audio covers the product need without a separate STT pipeline.

Contents

- [1. Roadmap at a Glance](#)
- [2. Sprint 2 — Organizational Archive & Search](#)
- [3. Sprint 3 — Jira & Operational Workflow](#)
- [4. Sprint 4 — Users, Workspaces & Calendar](#)
- [5. Sprint 5 — Scale, Quality & Deployment](#)
- [6. Sprint 6+ — Intelligent Organizational Product](#)
- [7. Advanced Capabilities \(Differentiators\)](#)
 - [1. Facilitator Guide](#)
 - [2. SOW / Contract Alignment](#)
 - [3. Sentiment Analysis](#)

8. [MLOps & the Quality Loop](#)
9. [Prioritization](#)
10. [Technical Dependencies](#)
11. [Longer-Horizon Research Ideas](#)

1. Roadmap at a Glance

Sprint 1 
MVP demo



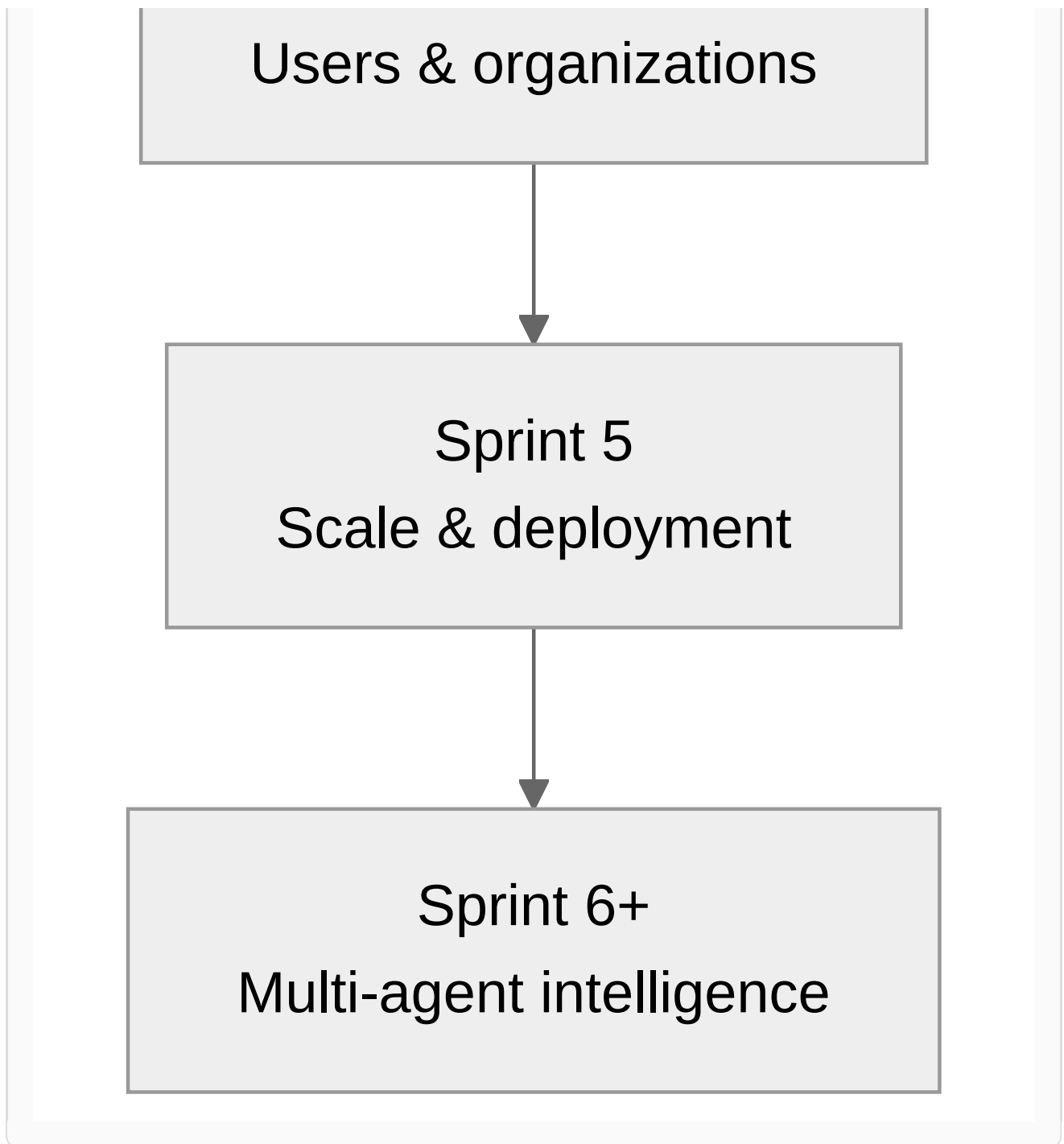
Sprint 2
Archive & multi-meeting
RAG



Sprint 3
Smart Jira & integrations



Sprint 4



The arc of the roadmap is a deliberate progression: from a single meeting (MVP) to an organizational memory (Sprint 2), to real operational workflow (Sprint 3), to a true multi-tenant product (Sprints 4–5), and finally to differentiated multi-agent intelligence (Sprint 6+).

2. Sprint 2 — Organizational Archive & Search

Goal: move from "one meeting" to "an organizational memory of meetings."

Capability	Description	Proposed implementation
------------	-------------	-------------------------

Multi-meeting RAG	Ask across many meetings, filtered by date, project, or participant.	Chroma metadata <code>org_id</code> , <code>project</code> , <code>date</code> ; filter at query time.
Archive page	List meetings with full-text search over title/summary.	SQLite FTS5 on <code>title</code> , <code>summary</code> .
Tags & projects	Categorize meetings (stand-up, planning, review).	<code>tags</code> , <code>project_key</code> columns in SQLite.
Summary export	Export to Markdown / PDF to share with managers.	Astro template or WeasyPrint from the same <code>MeetingAnalysis</code> .
Meeting comparison	"Last sprint's decisions vs. now" — an agent over multiple meeting ids.	<code>compare_agent</code> + top-k from each meeting.

Acceptance criteria

- Three meetings ingested; one cross-meeting question cited from ≥ 2 meetings.
- Archive search under 500 ms (up to ~100 meetings).
- One meeting's summary exported with no RTL rendering errors.

3. Sprint 3 — Jira & Operational Workflow

Goal: tasks actually reach the right people in Jira, not anonymous issues.

Capability	Description	Technical
Assignee mapping	"Sara" in the transcript → a Jira <code>accountId</code> .	<code>speaker_jira_map</code> table; settings UI (already scaffolded).
<code>jira_agent</code>	Preview, de-duplicate, suggest epic/link.	Pydantic-AI tools: <code>search_existing_issues</code> , <code>preview_create</code> .
Jira OAuth	Replace the personal API token — team-friendly.	Atlassian OAuth 2.0; encrypted refresh token storage.

Edit before send	Edit tasks in the UI, then bulk create.	Temporary state; POST only after confirmation.
Sync status	Show the issue key and Jira link on the meeting page.	<code>jira_keys_json</code> per task.
Issue templates	Project custom fields (story points, component).	Map from <code>JIRA_PROJECT_KEY</code> + a YAML config.

4. Sprint 4 — Users, Workspaces & Calendar

Goal: real multi-user use and connection to everyday tools.

Capability	Description	Technical
Authentication	Email/password login or organizational SSO.	JWT sessions, or OIDC (Keycloak / Google Workspace).
Workspaces	Each organization's data is isolated — meetings, speaker map, Jira.	<code>org_id</code> on every table; tenancy middleware.
Roles (RBAC)	admin · editor · viewer — who may create Jira issues.	<code>memberships</code> table.
Calendar (metadata)	Import title/time/attendees from Google Calendar — transcript still pasted/uploaded.	Read-only Calendar API; pre-fill the meeting form.
Summary email	Auto-send the summary + meeting link to attendees.	SMTP or Resend/Cloudflare Email; an RTL HTML template.
Slack/Teams alert	Short message: N new tasks + link.	Incoming webhook.

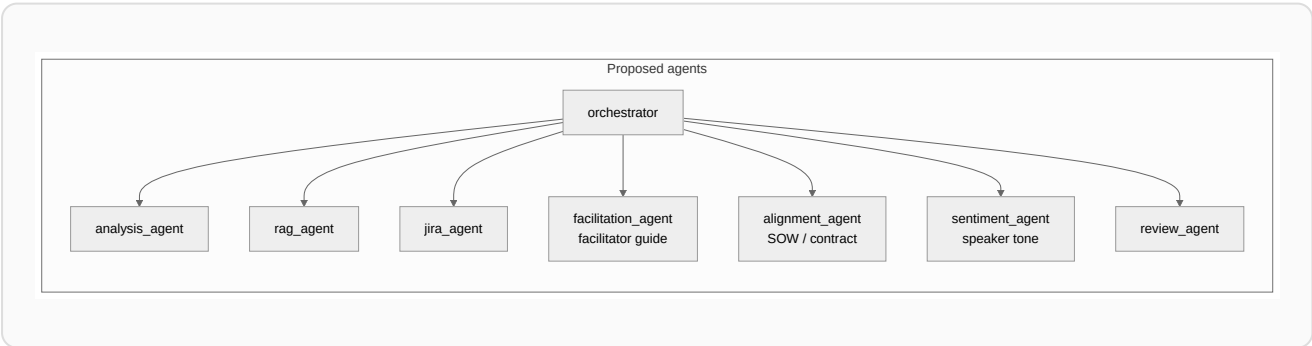
5. Sprint 5 — Scale, Quality & Deployment

Goal: ready for an organizational demo and production monitoring.

Capability	Description	Technical
PostgreSQL	Replace SQLite for multiple workers and backups.	SQLAlchemy + Alembic migration from the current schema.
Vector at scale	Thousands of chunks without slowdown.	pgvector or Chroma Cloud; a retention policy.
Ingest queue	Analyze long meetings in the background.	Redis + worker (Celery / ARQ); UI status polling.
Observability	Track Gemini latency and Jira errors.	OpenTelemetry + structured logs; a dashboard.
Deployment	Docker Compose; optional Cloud Run / VPS.	frontend + API + Postgres in one compose file.
Eval & prompts	A golden-transcript set; quality regression on summaries.	pytest + an LLM-judge score or a human rubric.
Rate limiting	Prevent API abuse.	Per-org rate limit; embedding cache.

6. Sprint 6+ — Intelligent Organizational Product

Goal: competitive differentiation — beyond "summary + Jira."

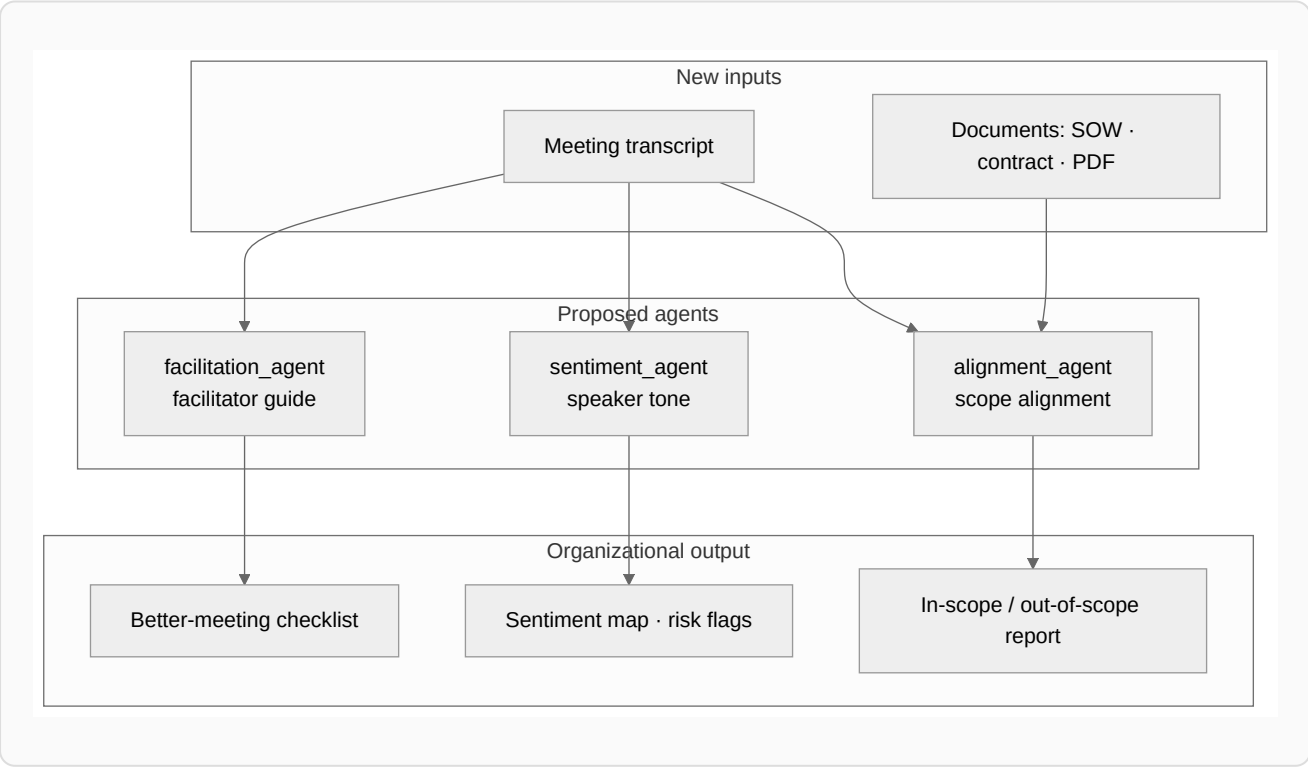


Idea	Value
Decision tracking	Follow decisions across meetings — "was decision X carried out?"

Action-item dashboard	All open tasks from all meetings, filtered by assignee.
Meeting templates	Prompts specialized for stand-up / retro / planning.
Participation analysis	Each speaker's share of talk time; warn on one-sided meetings.
Decision conflict	An agent that compares a new meeting against past decisions.
Follow-up suggestion	Auto-draft the next agenda from open tasks.
Confluence / Notion	Publish minutes to the organization's wiki.
Analysis versioning	Human edits to a summary; re-run only part of the pipeline.
Privacy	PII masking · retention · GDPR export.
On-prem	In-network deployment; optional self-hosted model.

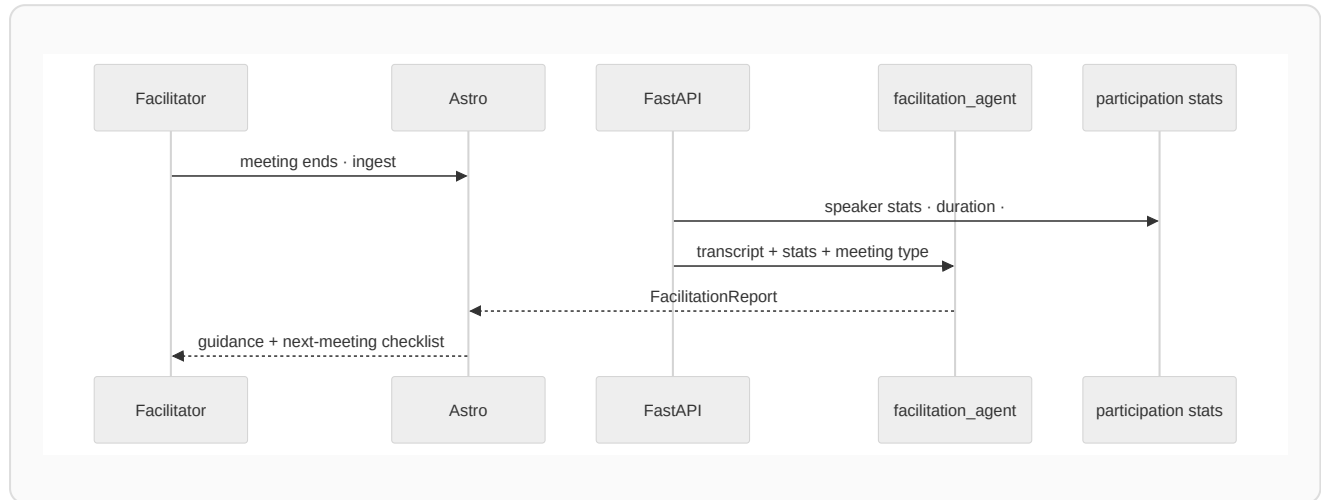
7. Advanced Capabilities (Differentiators)

Three capabilities turn the assistant from a note-taker into a meeting-intelligence product. Each adds a new agent and, where relevant, a new input.



7.1 Facilitator Guide (*implemented*)

After each meeting the system gives the **facilitator** concrete, coaching-style advice: what went well, where time was wasted, and how to run the next meeting shorter and more effectively.



Manual checklist (usable today, no AI)

Before the meeting

- ☐ State a one-line goal and the expected output (decision / task / information).
- ☐ Send an agenda with a rough timebox per item.
- ☐ Invite only the necessary people; make each person's role clear.
- ☐ Make reference documents (SOW, contract) available in advance.

During the meeting

- ☐ State decisions explicitly: "we decided that..." with an owner.
- ☐ Send off-topic threads to a "parking lot."
- ☐ Give a 30-second progress recap every 15–20 minutes.

After the meeting

- ☐ Publish the summary + tasks within 24 hours.
- ☐ Schedule a follow-up only if there is an open task.

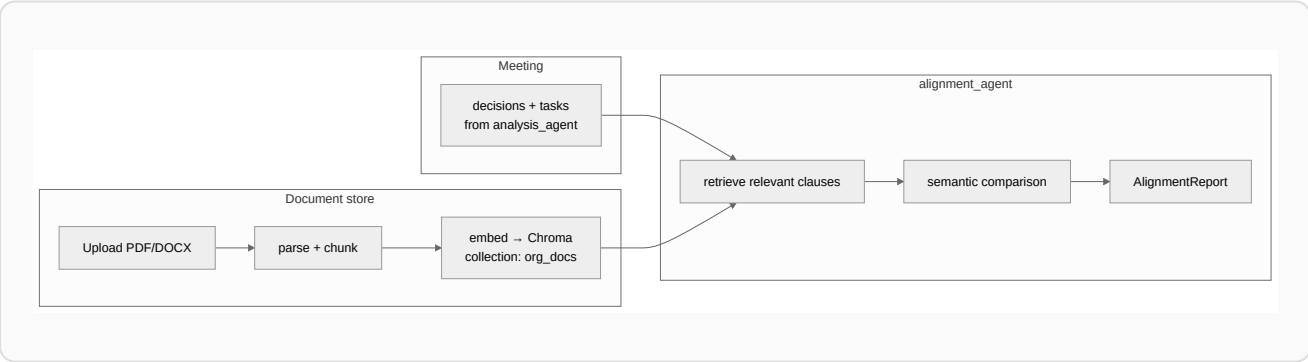
FacilitationReport output

Field	Description
what_went_well	2–3 positives (e.g. clear decisions, balanced participation).
improvements	Specific suggestions (e.g. "section X took 40% of the time with no decision").
next_meeting_agenda	A draft agenda for the next meeting, from open tasks.
timebox_suggestion	A shorter duration or a different format (e.g. async update).
coaching_summary	A short, encouraging summary for the facilitator.
facilitator_score	An optional 1–5 score against a rubric.

API: GET /api/meetings/{id}/facilitation — implemented, with a "Facilitator Guide" tab in the UI.

7.2 SOW / Contract Alignment (planned)

An organization can upload a **Statement of Work**, contract, or RFP. After each meeting, `alignment_agent` compares the meeting's decisions and commitments against the document's clauses and labels each as **in-scope**, **out-of-scope**, or **needs legal/manager approval**.



Proposed data model

Entity	Fields
OrgDocument	id, title, type (sow contract rfp), project_key, uploaded_at

AlignmentFinding	meeting_decision , doc_excerpt , status (aligned out_of_scope unclear), confidence , doc_id
AlignmentReport	A list of findings + a summary for the project manager.

Proposed API

Method	Path	Purpose
POST	/api/documents	Upload an SOW/contract · index in Chroma.
GET	/api/documents	List a project's documents.
POST	/api/meetings/{id}/alignment	Run alignment_agent · comparison report.

Product caveat: AI output does not replace legal counsel — for binding contracts, human approval is mandatory. The UI must show a clear disclaimer.

7.3 Sentiment Analysis (*planned*)

Over the **transcript text** (not raw audio), for each speaker and each phase of the meeting: overall tone (positive / neutral / negative), **incongruity** (e.g. positive words hiding a concern), and the trend of sentiment over the meeting.

Transcript turns
per speaker



Chunk by speaker
or time window



sentiment_agent
Gemini structured output

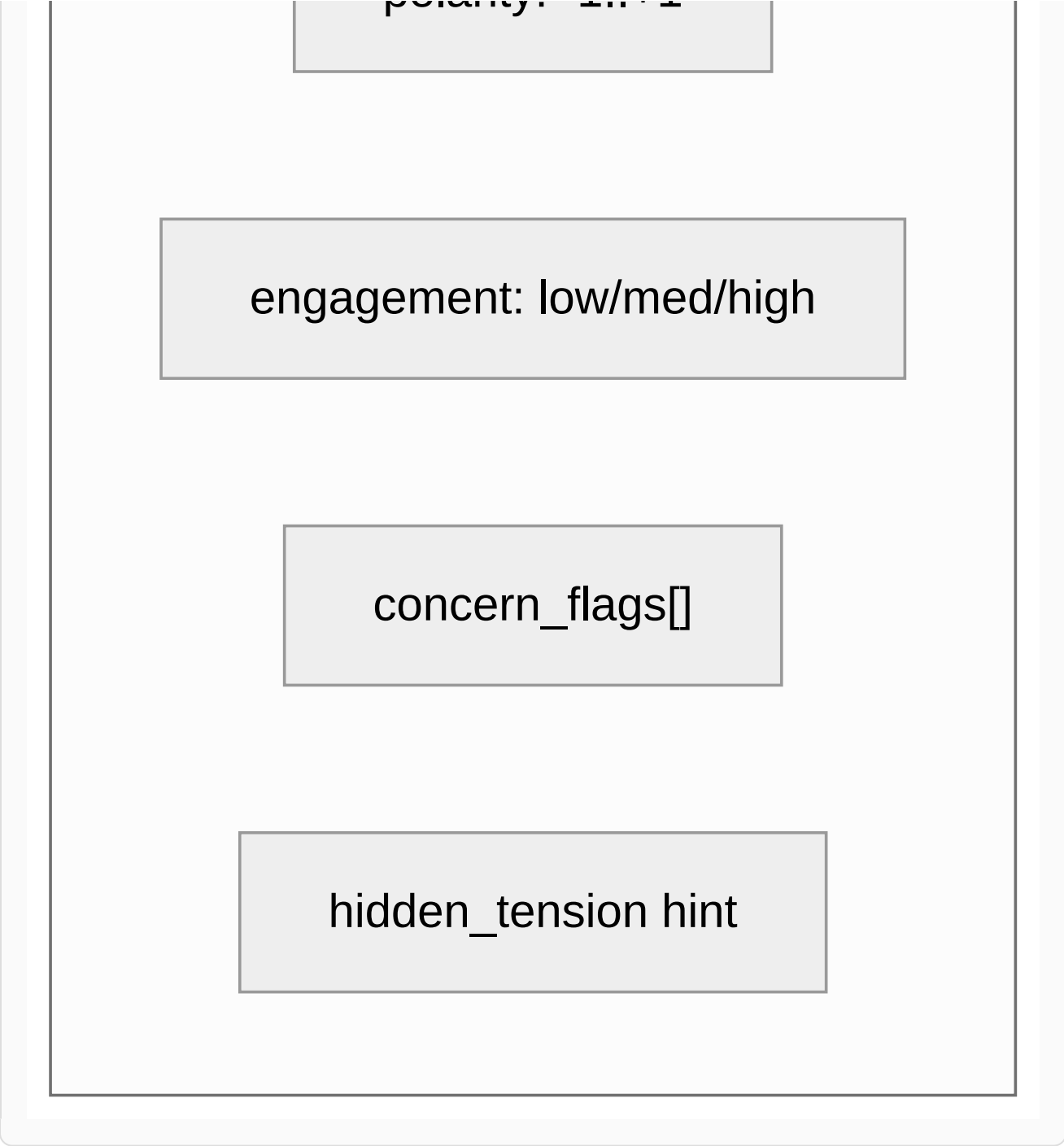


SpeakerSentimentProfile



Metrics

polarity: -1 +1



SpeakerSentimentProfile output

Field	Meaning
speaker	Speaker name from the transcript.
overall_tone	positive neutral negative mixed.
polarity_score	A number from -1 to +1.
concern_phrases	Short quotes that hint at concern or disagreement.

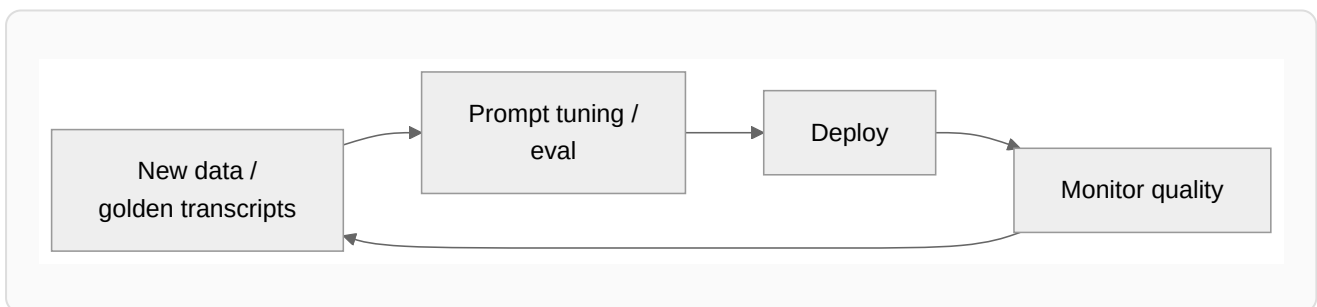
hidden_dissatisfaction	bool + explanation (e.g. "verbal agreement but stress on risk").
timeline	Tone change across the start / middle / end of the meeting.

Proposed API: GET /api/meetings/{id}/sentiment — a new tab beside summary/tasks/RAG.

Ethics & privacy: sentiment analysis for team management is sensitive. Only an admin/facilitator should see it; it must be organizational opt-in, with no negative labels in public reports. Framing in the UI: "a signal for a follow-up conversation," never "a personality judgment."

8. MLOps & the Quality Loop

The MVP does not retrain a model — it consumes a hosted LLM. The operational concern, therefore, is not model training but **quality assurance of prompts and outputs over time**, which is exactly where an AI product tends to regress silently.



- **Golden set:** a curated set of transcripts with reference summaries and expected tasks.
- **Automated eval:** score each new prompt/model version against the golden set with an LLM-judge and/or a human rubric, run as part of CI.
- **Prompt versioning:** treat prompts as versioned artifacts so a regression can be traced to a specific change.
- **Drift monitoring:** watch summary quality, RAG citation accuracy, and Jira conversion over time, and alert on degradation.

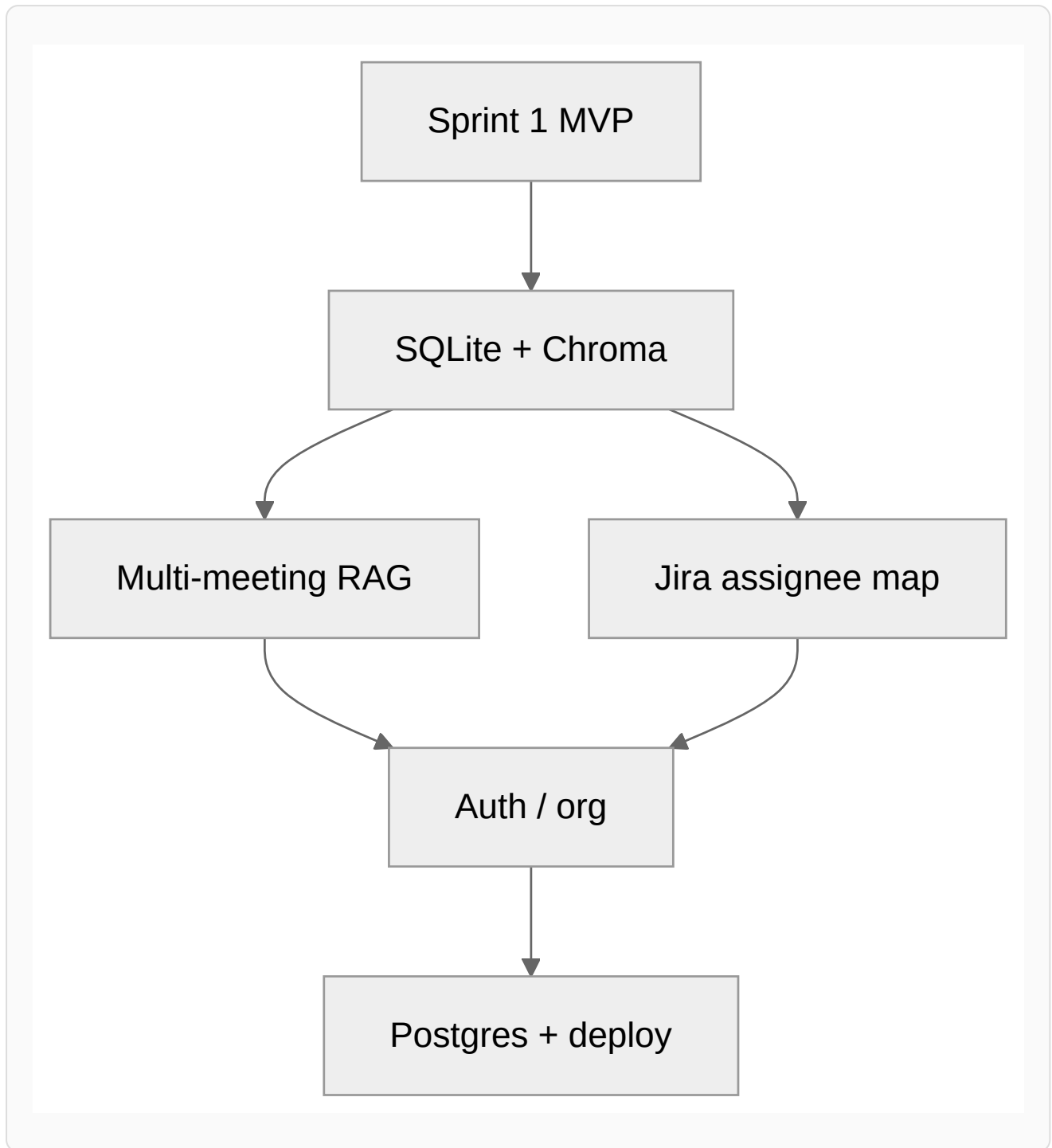
This is scheduled with Sprint 5 (Scale, Quality & Deployment) and is the honest counterpart to the AIOps/MLOps theory: the lifecycle the product would adopt as it matures, scoped

realistically rather than over-promised in the MVP.

9. Prioritization

Priority	Item	Rationale
P0	Multi-meeting RAG + FTS archive	Highest value over the MVP.
P0	Jira assignee mapping	A real pain point for operational teams.
P1	Auth + workspace	Prerequisite for an organizational customer.
P1	Edit-before-create for Jira	User trust in AI output.
P1	Facilitator guide (done) + deeper coaching	Immediate differentiation, light infrastructure.
P1	SOW/contract alignment	Value for PMs and fixed-scope contracts.
P2	PostgreSQL + queue	When traffic exceeds demo scale.
P2	Calendar metadata + summary email	Reduces data-entry friction.
P2	Sentiment analysis	Only after RBAC and ethical opt-in.
P3	Multi-agent · Confluence	Long-term differentiation.

10. Technical Dependencies



The dependency graph explains the sprint ordering: the storage foundation enables both multi-meeting RAG and assignee mapping; both of those motivate multi-tenancy (auth/org); and a real tenant load is what finally justifies PostgreSQL and a production deployment.

11. Longer-Horizon Research Ideas

Beyond the planned product work, several directions would make strong follow-on research or thesis extensions:

- **Cross-meeting decision graphs.** Model decisions as nodes and detect contradictions, reversals, and unfulfilled commitments across an organization's meeting history.
- **Multi-agent debate for summaries.** Use a critic/reviewer agent to challenge the first-pass summary, measuring whether adversarial review improves factual faithfulness.
- **Semantic, speaker-aware chunking.** Compare turn-based chunking against semantic and speaker-aware chunking on retrieval accuracy for Persian transcripts.
- **Faithfulness evaluation for Persian RAG.** Build an evaluation harness for citation accuracy and hallucination rate specific to low-resource-language meeting data.
- **Privacy-preserving analysis.** On-device or PII-masked pipelines so sensitive meetings can be analyzed without leaving the organization's network.
- **Human-in-the-loop quality.** Capture human edits to summaries and tasks as a feedback signal for prompt improvement and per-organization adaptation.

Related: [Project Overview & Vision](#) · [Technical Implementation](#).